E-COMMERCE WEBSITE WITH AI SUPPORT

Project work submitted to Thiruvalluvar University, Serkkadu, Vellore.

In Partial Fullfillment for the award of degree

BACHELOR OF COMPUTER APPLICATION

By

ARJUN K	20923U09017
ANNAMALAI K	20923U09016
ARJUN V	20923U09018
ANBARASU M	20923U09013
ANBU K	20923U09014

Under the Guidance of

Prof. Mrs.R.INDRA M.sc., B.ED.,

Assistant Professor,
Department of Computer Application

Shanmuga Industries Arts and Science College

Tiruvannamalai

DEPARTMENT OF COMPUTER APPLICATION

SHANMUGA INDUSTRIES ARTS & SCIENCE COLLEGE

Accredited with "B+" Grade by NAAC,

Certified Under Section 2(f) & 12B of the UGC Act 1956, An ISO 9001:2015 Certified Institution Permanently Affiliated to Thiruvalluvar University, Vellore Approved by Govt. of Tamilnadu & AICTE, Tiruvannamalai 606603

OCTOBER – 2025

E-COMMERCE WEBSITE WITH AI SUPPORT

Bonafide project work done by

ARJUN K	20923U09017
ANNAMALAI K	20923U09016
ARJUN V	20923U09018
ANBU K	20923U09014
ANBARASU M	20923U09013

In partial fullfillment of the requirement for the award of degree

BACHELOR OF COMPUTER APPLICATION

To

DEPARTMENT OF COMPUTER APPLICATION

SHANMUGA INDUSTRIES ARTS & SCIENCE COLLEGE

Accredited with "B+" Grade by NAAC, Certified
Under Section 2(f) & 12B of the UGC Act 1956,

An ISO 9001:2015 Certified Institution,
Permanently Affiliated to Thiruvalluvar University, Vellore – 632 115,
Approved by Govt. of Tamilnadu & AICTE, Tiruvannamalai – 606603.

Internal Guide **Head of the Department**

Submitted to the Viva - Voice Examination held on _____

Internal Examiner **External Examiner**

DECLARATION

We declare that the project entitled submitted to "E-COMMERCE WITH AI SUPPORT"

Department Computer Application, Shanmuga Industries Arts and Science

College, Tiruvannamalai in partial fulfillment of the requirement for the

award of the degree Bachelor of Computer Application for Thiruvalluvar

University, Vellore. This report was prepared on my own efforts and it has

not been produced earlier towards the award of any other degree or diploma.

Place:
Date:

Signature of the Candidate

1.) ARJUN K : _____

2.) ARJUN V : _____

3.) ANNAMALAI K : _____

4.) ANBU K : _____

5.) ANBARASU M : _____

CERTIFICATE

This is to certify that the project titled E-COMMERCE WEBSITE WITH AI SUPPORT the bonafide work carried student of Bachelor of(Computer application)of Shanmuga Industries arts and science collage during the academic year 2023-2026, in partial fullfillment of the requirements for the award of the degree of Bachelor of Science (Computer application) and that the project has not formed the basis for the award previously of any other degree, diploma, fellowship or any other similar title.

Place:

Date: Signature of Guide

ACKNOWLEDGEMENT

We Wish to acknowledge my indebtedness and deep sense of gratitude to the Honorable **Chairman Mr. M.N. PALANI**, Beloved **Secretary& Correspondent Mr. L. VIJAYANAND, B.Com.,** and Beloved **Treasurer Mr. A. SRIDHAR, B.Com.,** Shanmuga Industries Arts and Science College, Tiruvannamalai,

We like to express my gratitude to **Prof. Dr. K. ANANDARAJ, M.Sc., M.Phil., Ph.D., NET., Principal,** for his constant support and motivation throughout the project.

Also, we like to express my gratitude to **Prof. Dr. K. ANNAMALAI, M.Com., M.Phil., Ph.D., B.Ed., Vice Principal,** for his constant support and motivation throughout the project.

We were obliged and indebted to thank **Prof. Mr. V. SAKTHIVEL, M.C.A., M.Phil.,** Associate Professor **Head of Department of Computer Applicatio**n who helped to complete my project successfully.

We are very thankful to my **Guide Prof. Mrs. R. INDRA, M.Sc., B.Ed.,** Assistant Professor, of Computer application who helped me to complete the dissertation successfully. I am grateful to her for her encouragement, invaluable guidance throughout my project.

We also thank all faculties & support staff members of our college who had helped me in successful completion of this project.

Abstract

Introduction

AAA Shopping – Modern E-Commerce Platform Powered by MERN Stack project describes the end-to-end design and implementation of a comprehensive e-commerce web application built using the MERN (MongoDB, Express.js, React, Node.js) stack. The core objective is to provide users a seamless, secure, and user-friendly digital shopping experience—mirroring real-world online stores—through a robust full-stack architecture.

System Architecture

The application follows a full-stack client-server model:

- **Frontend:** Developed in React, responsible for UI, route management, user interactions, and API requests.
- **Backend:** Node.js/Express REST API serving data, processing orders, and enforcing business logic.
- **Database:** MongoDB, managing all product and order data persistently.
- **API Communication:** JSON over HTTP via well-structured endpoints.

Frontend

The frontend interface, developed with React and Vite, delivers a modern, responsive, and interactive user experience. Customers can explore a dynamic catalog of products, view details, add products to their shopping cart, and place orders instantly. Visual components are modular and built with attention to UI/UX best practices, such as quick-view, “Buy Now” actions, and customer-centric banners with satisfaction and service guarantees.

backend

The backend is engineered with Express and Node.js to handle business logic, API routing, and data processing. MongoDB, accessed via Mongoose, stores all persistent data—including products, orders, and user actions. RESTful APIs permit efficient retrieval of product lists and secure creation of orders, while modular routing (e.g., /api/v1/products, /api/v1/order) keeps the codebase maintainable and extensible. CORS middleware and structured error handling ensure smooth development, robust security, and effective communication between frontend and backend.

The project emphasizes reliability and trust through features such as delivery assurance, satisfaction guarantees, and responsive customer support messaging. The implementation follows MVC principles to separate concerns, streamline development, and facilitate future enhancements such as authentication, payments, and analytics.

conclusion

This work not only demonstrates technical mastery in the MERN stack but also models best practices in real-world e-commerce deployment. It is designed for students, educators, and developers seeking a scalable, maintainable, and industry-relevant foundation for online retail applications, serving both as an academic reference and practical solution for modern businesses.

S.no	CONTENTS	PAGE NO:
1.	CHAPTER 1 **Introduction** 1.1 overview of website 1.2 System Architecture 1.3 Technology use with explanation: 1.3.1 frontend 1.3.2 backend	
2.	CHAPTER 2 **API** 2.1 Products API 2.2 Order API 2.3 Summary Table	
3.	CHAPTER 3 **Designs** 3.1Frontend Designs 3.2 Backend Designs 3.3 Database Designs 3.3.1 Order Model 3.3.2 Product Model	
4.	CHAPTER 4 **KEY FEATURES** 4.1 Order Process Flow 4.2 Security & Users Assurances 4.3 Testing & Deployment	
5.	CHAPTER 5 **WEBSITE AI SUPPORT** 5.1 Ai Support(Powered by Chatling) 5.2 Purpose & Features 5.3 Chatling Platform & Customisation 5.4 Integration & User Experience 5.5 Benefits	

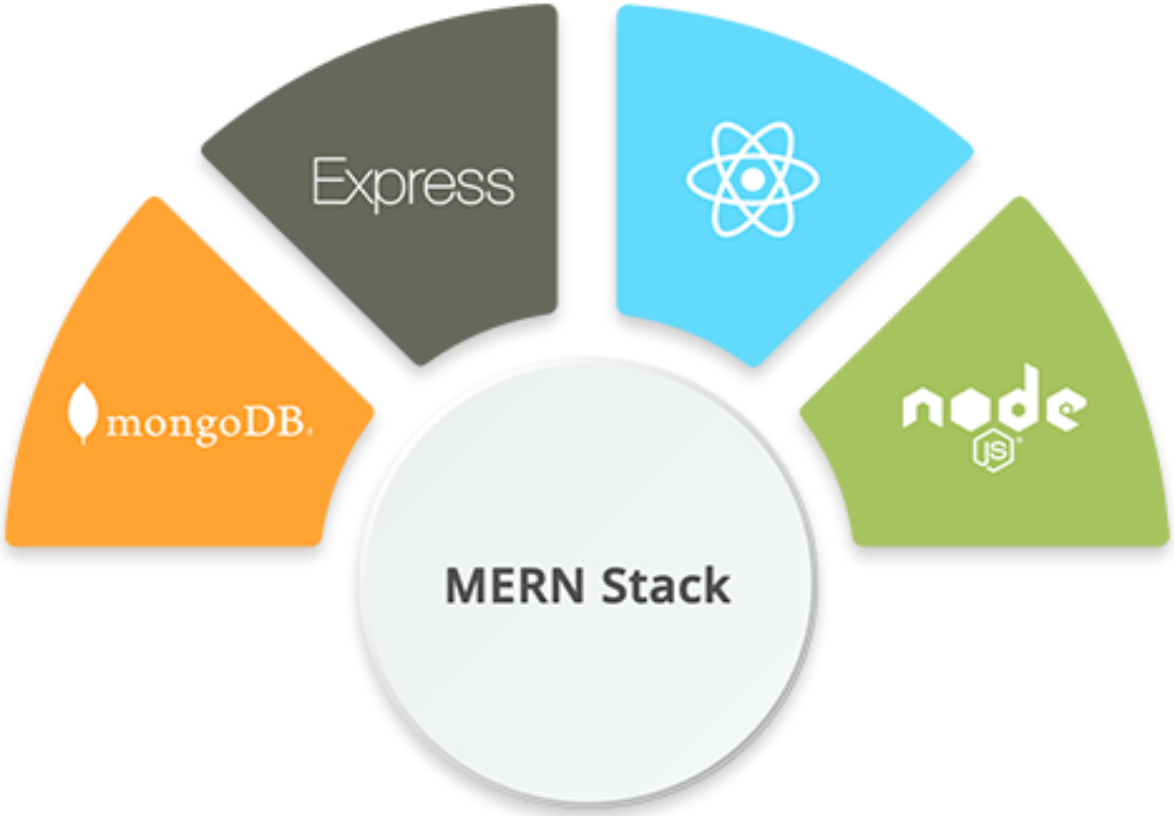
6.	CHAPTER 6 **SOURCE CODE** 6.1 Screenshots 6.2 Code 6.3 Future Enhancement Possibilities 6.4 Conclusion	

Chapter 1

INTRODUCTION

1.1Overview of website

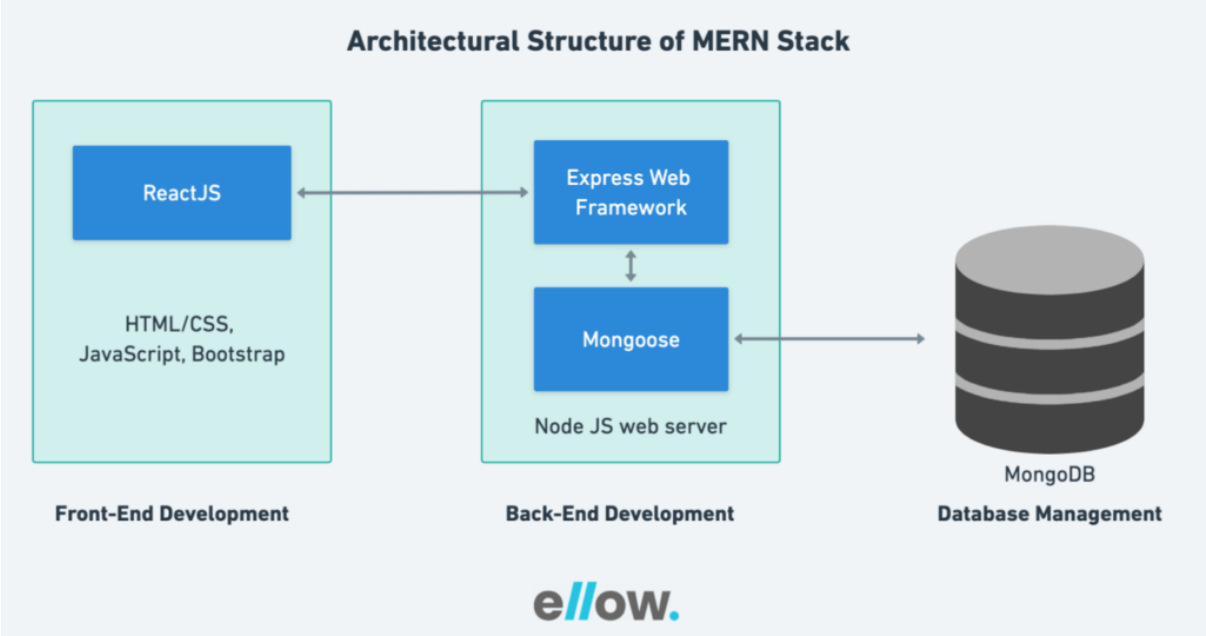
This e-commerce project is a complete web application developed using the MERN stack (MongoDB, Express.js, React, Node.js), designed to provide a modern, secure, and efficient online shopping experience. Users can browse an interactive catalog of products, add items to their cart, and seamlessly place orders through a responsive React frontend that communicates with an Express REST API backend. All product and order data is persistently stored in a MongoDB database, ensuring accuracy and scalability. The backend architecture employs modular routing, controller logic, and robust error handling to support reliable operations. Additionally, features like satisfaction and delivery guarantees are built in to boost user trust and service quality. The system's design prioritizes clean code organization, maintainability, and extensible infrastructure, making it valuable for academic demonstration and real-world deployment in digital commerce.



1.2 System Architecture

The application follows a full-stack client-server model:

- **Frontend:** Developed in React, responsible for UI, route management, user interactions, and API requests.
- **Backend:** Node.js/Express REST API serving data, processing orders, and enforcing business logic.
- **Database:** MongoDB, managing all product and order data persistently.
- **API Communication:** JSON over HTTP via well-structured endpoints.
- **Chatbot:** Chatling empowered chatbot created by UI/UX.



1.3 Technology use with explanation:

1.3.1 Frontend:

The frontend of this e-commerce project is built using React, delivering a fast, interactive, and highly responsive user experience. Leveraging the component-based architecture of React, the UI is organized into modular pieces such as product listings, product cards, cart preview, and checkout forms, each designed for reusability and maintainability. Users can browse products, view detailed descriptions, add items to their cart, and place orders—all within a single-page application that updates content dynamically without full page reloads. State management ensures that cart operations, product selections, and user actions are instantly reflected in the interface for a smooth shopping flow. React Router is used to manage navigation, while Bootstrap or custom CSS provides consistent styling and responsive design suitable for mobile and desktop devices. Fetch requests are used by the frontend to interact with backend APIs for product data and order submissions.

1.3.2 Backend:

The backend of this project is developed using Express.js, which is a powerful and lightweight Node.js web framework designed to handle server-side logic for modern web applications. Express allows the creation of efficient REST APIs by managing HTTP requests and responses with clean, modular route definitions. In this e-commerce application, the backend exposes APIs that enable the frontend to fetch product listings from the database and process new orders submitted by users. Core responsibilities include connecting to the MongoDB database to store and retrieve products and orders, ensuring smooth and secure data communication, and handling business logic such as order validation and error management. Express also supports middleware for features like JSON parsing and cross-origin requests (CORS), making the server robust and development-friendly. Overall, Express.js provides the backbone for reliable, scalable, and maintainable back-end operations in the project.

CHAPTER-2

API

2.1 Products API

GET /api/v1/products

- Purpose: Fetches the complete list of products available in the store.
- Functionality: When this endpoint is called, the backend queries the MongoDB products collection using Mongoose and returns an array of product objects. Each object contains information like product name, price, description, images, and rating.
- Usage: This API is consumed by the frontend to display product listings in the catalog and detailed product pages.

2.2 Orders API

POST /api/v1/order

- Purpose: Allows users to place an order with selected products.
- Functionality: The frontend sends a POST request to this endpoint with the cart items, order amount, status, and timestamp in the request body. The backend validates the data, then creates and saves a new order document in the MongoDB orders collection using Mongoose.
- Usage: This API is triggered when users complete checkout and submit their orders. On success, it responds with confirmation and order details; on error, it provides a failure message.

2.3 Summary Table

Endpoint	Method	Description	Returns
/api/v1/products	GET	Fetch all product data	Array of product objects
/api/v1/order	POST	Record a new user order in the database	Success/failure message, order details
/api/v1/orders	POST	Alternative endpoint for order creation	Success/failure message

Other: REST principles, JavaScript/JSX, Git for version control

---

Chapter 3

Designs

3.1Frontend Design

The React frontend is organized with modular components for scalability:

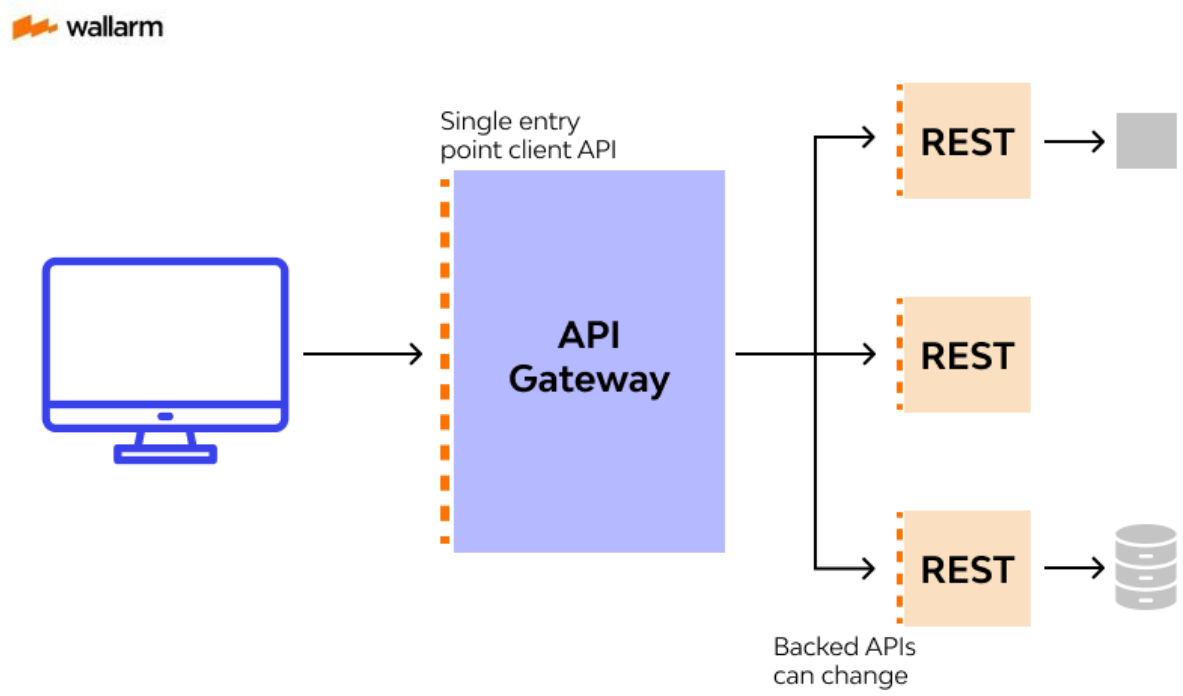
- **Product Components:** View, list, and purchase products
- **Order Components:** Initiate and confirm orders
- **UI Elements:** Header, footer, about, service containers, and guarantee banners
- **State Handling:** Manages product/cart/order data, shows alerts on success/failure

API calls are handled via fetch, posting payloads (cart data, amount, etc.) to the backend.

3.2 Backend Design & API Endpoints

Express.js drives RESTful APIs. Major endpoints:

- **/api/v1/products** – GET: List all products
- **/api/v1/order** – POST: Place an order



Backend logic is divided into controllers (business logic) and routers (HTTP path handling), enforcing clean code and easy extension.

3.3 Database Design

Using Mongoose, two main schemas:

3.3.1 Product Model: Stores product info (name, description, price, images, etc.)

```
const mongoose=require('mongoose');

const productSchema= new mongoose.Schema({
    name: String,
    price: String,
    description: String,
    ratings: String,
    images:[

        {
            Image: String

        }
    ],
    category: String,
    seller: String,
    stock:String,
    numOfReviews:String,
    createdAt: Date
});

const productModel= mongoose.model('product',productSchema);

module.exports=productModel;
```

3.3.2 Order Model: Stores array of cart items, order total, status, and timestamp

Sample Order schema:

```
const mongoose=require('mongoose');
const orderSchema = new mongoose.Schema({
    cartItems: Array,
    amount: String,
    status: String,
    createAt: {type:Date,default:Date.now}
})

const orderModel=mongoose.model('order',orderSchema);

module.exports=orderModel;
```

CHAPTER 4

Key Features

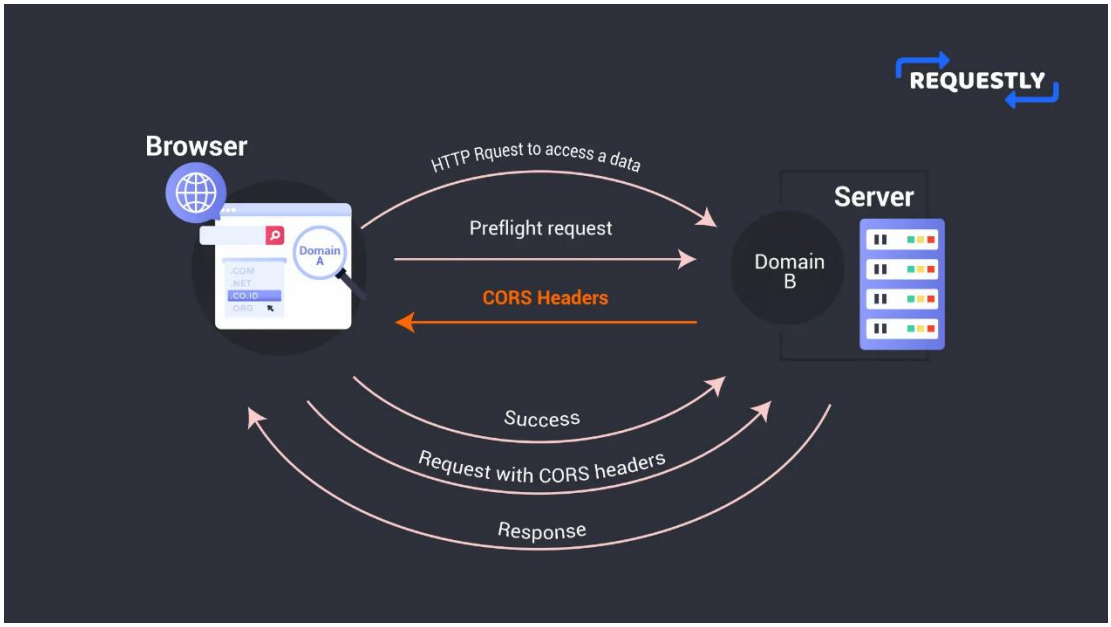
- Product listing, detailed product display, and search/filter
- Add to cart and direct "Buy Now" functionality
- Order creation and backend storage
- Service and satisfaction guarantees to promote trust
- Modular routes, error handling, and CORS support for development
- Extensible architecture for new features (payments, auth, analytics)

4.1 Order Process Flow

1. User clicks “buy now” on the product page.
2. Frontend sends POST request with order details to backend (/api/v1/order).
3. Backend validates, processes, and stores order in MongoDB.
4. API responds with status and full order data.
5. Frontend shows alert for success or failure.

4.2 Security & User Assurance

- CORS enabled for frontend-backend communication during development
    - Integrated satisfaction and delivery guarantees build user trust
- Error handling for API requests provides feedback
- Consideration for future features: authentication, input validation, secure payments



4.3 Testing and Deployment

- Manual UI/UX testing on local (Vite) and production builds
- Backend tested with Postman/cURL for endpoint validation
- Deployment-ready for platforms like Vercel, Heroku, or own serverchat

CHAPTER 5

WEBSITE AI SUPPORT

5.1 Website Chatbot (Powered by Chatling)

A key enhancement of this e-commerce project is the integration of an AI-powered website chatbot created with Chatling. The chatbot is designed to improve customer experience by offering instant and intelligent support to users while they navigate the site or encounter questions about products, orders, or services.

5.2 Purpose and Features

The chatbot serves multiple functions including:

- Answering frequently asked questions about products, shipping, and payment options.
- Guiding users through the shopping and order process.
- Offering information about ongoing deals, service guarantees, and store policies.
- Providing support if a user experiences any technical difficulty or requires help navigating the site.

5.3 Chatling Platform and Customization

Chatling enables the creation and deployment of website chatbots with minimal or no code. The bot for this project was trained using product information from the website, supporting documents, and a Q&A knowledge base, making its responses relevant and context-aware. Chatling's visual builder allowed the conversational flow to be mapped with different response blocks and user input handling, ensuring smooth and logical conversations.

5.4 Integration and User Experience

The chatbot is embedded directly into the website interface, appearing as a chat widget on all pages. Users can initiate chats at any point, with real-time responses generated by Chatling's AI or knowledge base. The solution is responsive and accessible on

desktop and mobile devices, without disrupting the shop's navigation or checkout flows.

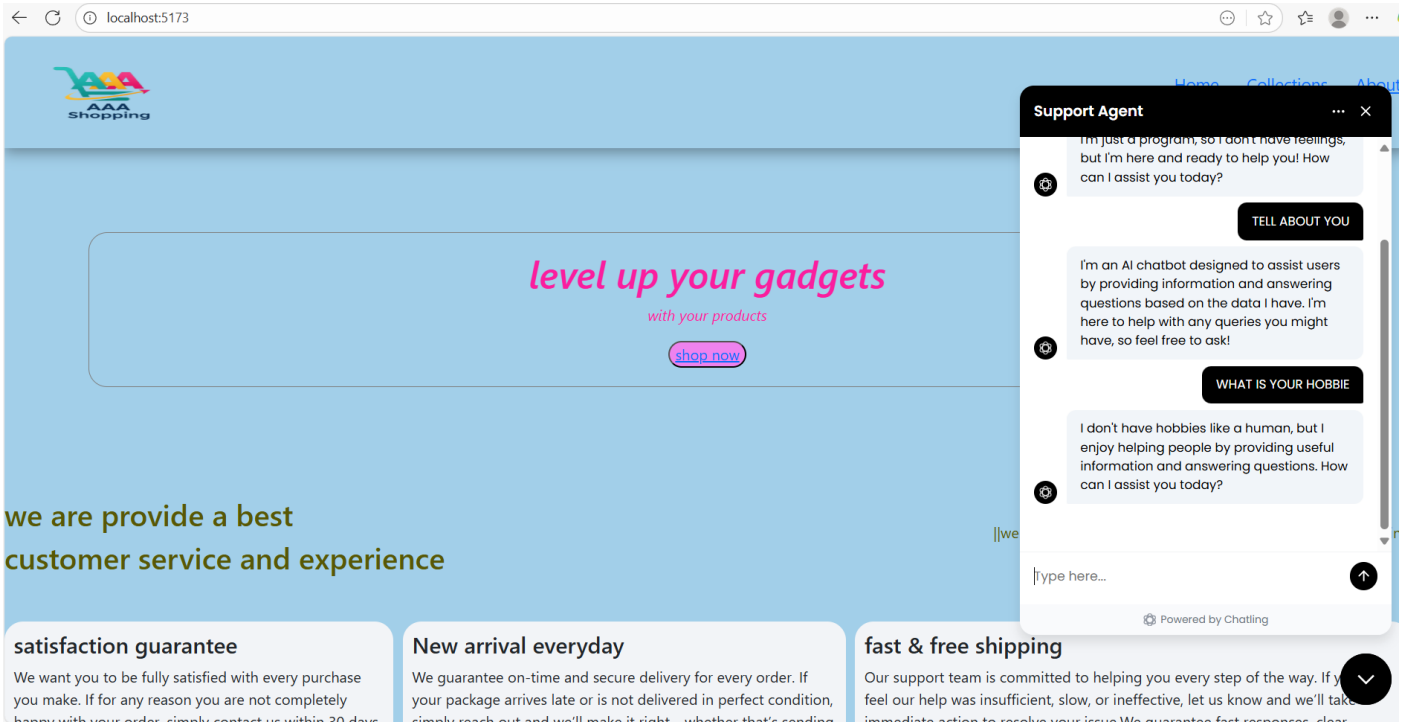
5.5 Benefits

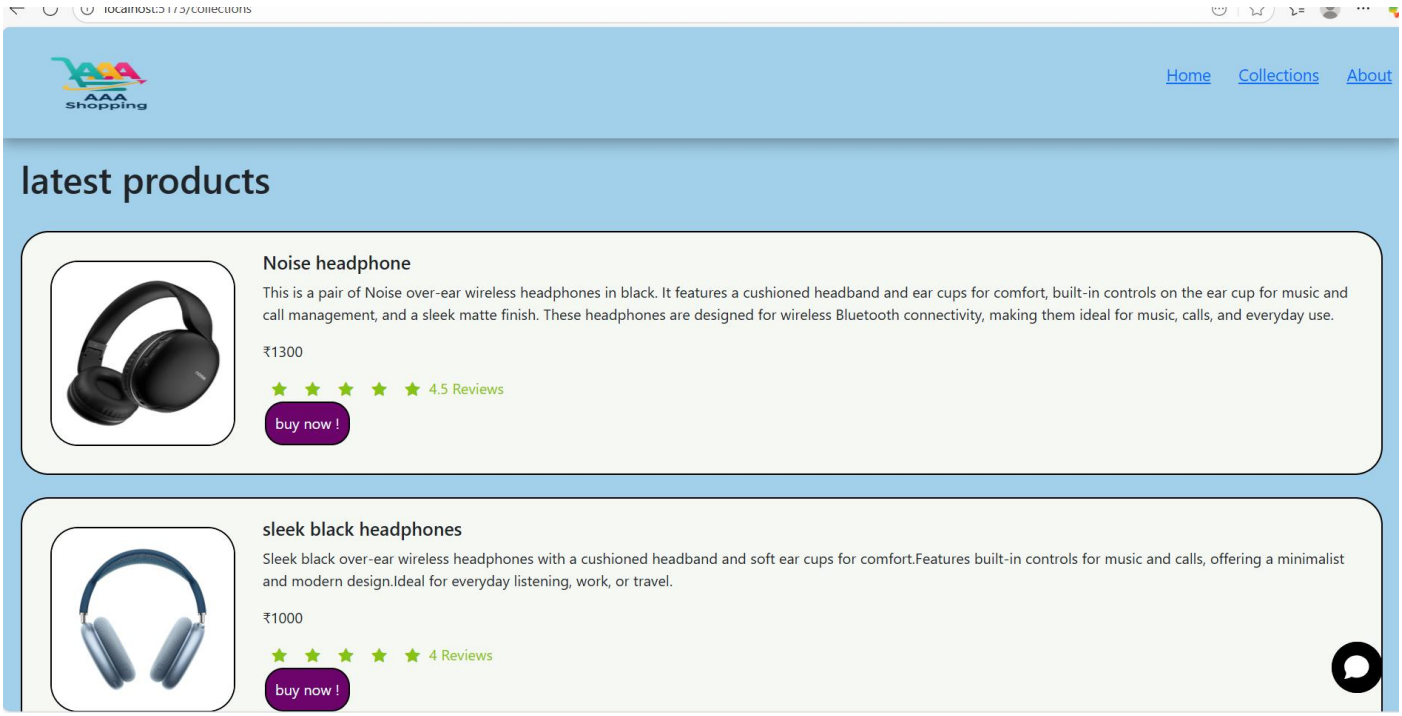
- Reduces support burden on staff by handling common user queries autonomously.
- Increases customer satisfaction through instant, always-on assistance.
- Builds trust and encourages engagement thanks to reliable, accurate information.
- Demonstrates the adoption of modern, AI-driven customer service technologies within the project.

Chapter 6

Source code

6.1 Screenshots:





6.2 CODE:

Frontend:

Logo:

Index.html:

```
<!doctype html>

<html lang="en">
```

```
<head>
  <meta charset="UTF-8" />
  <link rel="icon" type="image/svg+xml" href="/aaalogo1.png" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet"
    integrity="sha384-
sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossorigin="anonymous">

  <title>AAA-shopping</title>
</head>

<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-
FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
    crossorigin="anonymous"></script>
  <script> window.chtlConfig = { chatbotId: "7466811144" } </script>
  <script async data-id="7466811144" id="chtl-script" type="text/javascript"
    src="https://chatling.ai/js/embed.js"></script>
</body>

</html>
```

App.jsx:

```
import React, { Fragment } from "react";

import Home from "./pages/home";

function App(){
  return(
    <Fragment>
      <Home/>
      </Fragment>

  );
}

export default App;
```

main.jsx:

```
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import {createBrowserRouter,RouterProvider}from 'react-router-dom'
```

```
import About from './pages/About.jsx'
import Collections from './pages/Collections.jsx'

const router = createBrowserRouter([
  { path: '/', element: <App /> },
  { path: '/about', element: <About /> },
  { path: '/collections', element: <Collections /> }

]);

createRoot(document.getElementById('root')).render(
  <RouterProvider router={router} />
);
```

Header.jsx:

```
import React, { Fragment } from 'react'


function Header() {
  return (
    <Fragment>
      <header>

       <div className="navbar">
         <img src="/aaalogo1.png" alt="" />
```

```
        <div className="navbar-links">
          <p className="navbar-link"><a href="/">Home</a></p>
          <p className="navbar-link"><a href="/collections">Collections</a></p>

          <p className="navbar-link"><a href="/about">About</a></p>
        </div>

      </div>



    </header>
    </Fragment>
  )
}

export default Header
```

Footer.jsx:

```
import React, { Fragment } from 'react'
import 'bootstrap-icons/font/bootstrap-icons.css';

function Footer() {
  return (
    <Fragment>
      <footer className="footer">
        <p>© 2025 AAA-shopping. All rights reserved.</p>
```

```
<p>
  Contact us: <a href="mailto:support@aaa-shopping.com">support@aaa-shopping.com</a> | +91-12345-67890
</p>
<p>
  Subscribe to our Newsletter for exclusive deals!{" "}
  <button className="subscribe-button" onClick={() => alert("Subscribe clicked!")}>
    Subscribe
  </button>
</p>
<div className="cta-links">
  <a href="/collections" className="link">New Arrivals</a>
  <a href="/collections" className="link">Best Sellers</a>
  <a href="/collections" className="link">Clearance</a>
</div>
<div className="social-icons">
  <a href="https://facebook.com/aaashopping" target="_blank" rel="noreferrer" className="btn btn-outline-primary me-2">
    <i className="bi bi-facebook"></i>
  </a>
  <a href="https://wa.me/qr/JC4JYZKJWWWOI1" target="_blank" rel="noreferrer" className="btn btn-outline-info me-2">
    <i className="bi bi-whatsapp"></i>
  </a>
  <a
href="https://www.instagram.com/aaa_technology_03?igsh=MWszNG5jbzN6ZHBtOA==" target="_blank" rel="noreferrer" className="btn btn-outline-danger me-2">
    <i className="bi bi-instagram"></i>
```

```
        </a>
        <a href="https://www.linkedin.com/in/arjun-k-a91b58365?utm_source=share&utm_campaign=share_via&utm_content=profile&utm_medium=android_app" target="_blank" rel="noreferrer" className="btn btn-outline-primary me-2">
          <i className="bi bi-linkedin"></i>
        </a>
      </div>
    </footer>
  </Fragment>
 )
}

export default Footer
```

mainproducts.jsx:

```
import React, { Fragment } from 'react';

function mainProducts({ product }) {

 const handleBuyNow = async () => {
  try {
   const cartItems = [
    {
     product: {
      _id: product._id,
      name: product.name,
      price: product.price
```

```
      },
      qty: 1
    }
  ];
  const amount = Number(product.price * 1).toFixed(2);
  const status = 'pending';

  const response = await fetch('http://localhost:3000/api/v1/order', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      cartItems,
      amount,
      status,
      createdAt: Date.now()
    })
  });

  if (response.ok) {
    alert('Order placed successfully!.Thankyou.');
  } else {
    alert('Failed to place order. try again');
  }
 } catch (error) {
  alert('Error placing order: ' + error.message);
 }
};
```

```
 return (
  <Fragment>
   <div className="new-arrival">
    <div className="new-arrival-container">
     <div className='product-images'>
      <img src={product.images[0].Image} alt={product.name} />
     </div>
     <div className="details">
      <div className="card-title">
       <h5>{product.name}</h5>
       <p>{product.description}</p>
       <p>₹{product.price}</p>
      </div>
      <div className="ratings">
       <i className="bi bi-star-fill"></i>
       <i className="bi bi-star-fill"></i>
       <i className="bi bi-star-fill"></i>
       <i className="bi bi-star-fill"></i>
       <i className="bi bi-star-fill"></i>
       <span>{product.ratings} Reviews</span>
      </div>
      <div className="card-text"></div>
      <button onClick={handleBuyNow} className="btn view-btn">buy now
!</button>
     </div>
    </div>
```

```
    </div><br />
   </Fragment>
  );
}

export default mainProducts;
```

Home.jsx:

```
import React, { Fragment, useEffect, useState } from 'react';
import Header from '../components/header';
import Footer from '../components/footer';
import Service from '../Service';
import MainProducts from '../components/mainProducts';

function Home() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
   fetch('http://localhost:3000/api/v1/products')  // adjust URL to your products API
    .then((res) => res.json())
    .then((data) => {
      console.log('Fetched products:', data);  // check what API returns
```

```
      setProducts(data.products || []);  // adjust depending on your API response
structure
    })
    .catch((error) => {
      console.error('Error fetching products:', error);
    });
  }, []);

  if (products.length === 0) {
    return <p>No products available</p>;
  }

  return (
    <Fragment>
      <Header />

      <div className='homescreen'>
          <h1>level up your gadgets</h1>
          <p>with your products </p>
          <button><a href="/collections">shop now</a></button>

      </div>
     <Service />
    <div className="product">
      <h1>latest products</h1><br />
      {products.map((product) => (
      <MainProducts key={product._id} product={product} />
```

```
    ))}
     <br/>


      <Footer/>
    </div>
    <br />


   </Fragment>
  );
}


export default Home;
```

Services.jsx:

```
import React, { Fragment } from 'react'




function Service() {
   return (
      <Fragment>
         <div className="service">
            <div className="service-container-1">
               <div>
                  <h2>we are provide a best </h2>
                  <h2>customer service and experience</h2>
```

```
    </div>
    <div>
      <p>||we ensure that our customer have best satisfactional shopping </p>
    </div>

  </div>
  <div className="service-container-2">
    <div>
      <i className="fa-regular fa-face-smile"></i>
      <h4>satisfaction guarantee</h4>
      <p>We want you to be fully satisfied with every purchase you make. If for any reason you are not completely happy with your order, simply contact us within 30 days of receiving your product for a full refund, replacement, or exchange.
</p>
    </div>

    <div>
      <i className="fa-regular fa-face-smile"></i>
      <h4>New arrival everyday</h4>
      <p>We guarantee on-time and secure delivery for every order. If your package arrives late or is not delivered in perfect condition, simply reach out and we'll make it right—whether that's sending a replacement, refunding your order, or offering exclusive compensation.</p>

    </div>

    <div>
```

```
                <i className="fa-regular fa-face-smile"></i>

                <h4>fast & free shipping</h4>

                <p>Our support team is committed to helping you every step of the way. If you ever feel our help was insufficient, slow, or ineffective, let us know and we'll take immediate action to resolve your issue.We guarantee fast responses, clear solutions, and friendly service—your satisfaction with our support is 100% assured or your money back</p>

              </div >

            </div >


          </div>




   </Fragment>

  );

}


export default Service
```

collections.jsx:

```
import React, { Fragment, useEffect, useState } from 'react';

import Header from '../components/header';

import Footer from '../components/footer';


import MainProducts from '../components/mainProducts';
```

```
function Collections() {
  const [Products, setProducts] = useState([]);

  useEffect(() => {
    fetch('http://localhost:3000/api/v1/products')
      .then(res => res.json())
      .then(res => setProducts(res.products));
  }, []);

  const handleBuyNow = async () => {
    try {
      const cartItems = [
        {
          product: {
            _id: product._id,
            name: product.name,
            price: product.price
          },
          qty: 1
        }
      ];
      const amount = Number(product.price * 1).toFixed(2);
      const status = 'pending';

      const response = await fetch('http://localhost:3000/api/v1/order', {
        method: 'POST',
```

```
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        cartItems,
        amount,
        status,
        createdAt: Date.now()
      })
    });

    if (response.ok) {
      alert('Order placed successfully!');
    } else {
      alert('Failed to place order.');
    }
  } catch (error) {
    alert('Error placing order: ' + error.message);
  }
};

return (
  <Fragment>
    <Header />

    <div className="product">
      <h1>latest products</h1><br />
      {Products.map(product => (
        <MainProducts key={product._id} product={product} />
```

```
      ))}<br/>
    </div>
    <br />
    <Footer />
  </Fragment>
 );
}

export default Collections
```

About.jsx:

```
import React, { Fragment } from 'react'
import Header from '../components/header'
import Footer from '../components/footer'

function About() {
  return (
    <Fragment>
      <Header />
      <div>
        <div className="about-container">
          <h1 className="about-header">About Us</h1>
          <div className="about-content">
            <img
              src="/aboutimage.png"
              alt="About Illustration"
```

```
                    className="about-image"
                  />
                  <div className="about-text-container">
                    <p className="about-paragraph">
                      Welcome to our website! We are passionate about delivering quality
                      web solutions that empower businesses and individuals. Our team
                      works hard to ensure a seamless user experience and innovative
                      design.
                    </p>
                    <ul className="about-list">
                      <li>Customer Satisfaction Oriented</li>
                      <li>Innovative and Creative Solutions</li>
                      <li>Reliable and Efficient Support</li>
                      <li>Continuous Improvement & Learning</li>
                      <li>Contact us: +91 9361269396/8489574414</li>
                    </ul>
                  </div>
                </div>
              </div>



            </div>
            <Footer />
          </Fragment>
        )
      }
```

```
export default About
```

backend:

order.js:

```
const express= require('express');

const { createOrder } = require('../controllers/ordercontroller');
const router= express.Router();

router.route('/').post(createOrder);

module.exports=router;
```

product.js:

```
const express= require('express');
const { getProducts} = require('../controllers/productController');
const router= express.Router();

router.route('/products').get(getProducts);

module.exports= router;
```

ordermodule.js:

```
const mongoose=require('mongoose');
const orderSchema = new mongoose.Schema({
   cartItems: Array,
   amount: String,
   status: String,
   createAt: {type:Date,default:Date.now}
})

const orderModel=mongoose.model('order',orderSchema);

module.exports=orderModel;
```

productmodule.js:

```
const mongoose=require('mongoose');

const productSchema= new mongoose.Schema({
   name: String,
   price: String,
   description: String,
   ratings: String,
   images:[

      {
         Image: String
```

```
      }
    ],
    category: String,
    seller: String,
    stock:String,
    numOfReviews:String,
    createdAt: Date
  });

  const productModel= mongoose.model('product',productSchema);

  module.exports=productModel;
```

ordercontroller.js:

```
const orderModel=require('../modules/orderModel');

//create order
exports.createOrder=async(req,res,next)=>{
    const cartItems=req.body.cartItems||req.boby;

    const amount=Number(cartItems.reduce((acc,item)=>(acc + item.product.price *
item.qty),0)).toFixed(2);
    const status='pending';

    const order=await orderModel.create({cartItems,amount,status});
    res.json({
        success:true,
```

```
    order
  })
}
```

Productcontroller.js:

```
const productModel=require('../modules/product_module');

//get product api-/api/v1/product
exports.getProducts=async (req,res,next)=>{
  const products=await productModel.find({});

  res.json({
    success: true,
    products
  })
}
```

6.3 Future Enhancement Possibilities

- User authentication (JWT, OAuth)
- Admin dashboard (add/remove products)
- Payment integration (Stripe, Razorpay, etc.)
- Order history and real-time notifications
- Product reviews, advanced analytics

6.4 Conclusion

This MERN stack e-commerce project provides a scalable, secure, and modular starting point for modern online shopping platforms. The architecture supports easy expansion and real-world deployment, demonstrating best practices in full stack web development.

The completion and evaluation of this MERN stack e-commerce project showcase a comprehensive understanding of modern web development practices and technological integration. By leveraging MongoDB for data persistence, Express.js for robust server-side logic, React for dynamic user interfaces, and Node.js for seamless backend operations, the project demonstrates how these frameworks can work together to deliver a scalable, efficient, and engaging online shopping platform. The frontend presents products in a visually appealing, responsive format, enabling users to browse, select, and purchase items with ease. State management within React ensures that cart operations and order updates are reflected instantly, providing a smooth and interactive shopping experience across devices.

Communication between frontend and backend is facilitated through RESTful APIs, allowing real-time product retrieval and secure order processing. Products and orders are stored in MongoDB collections, managed via Mongoose, which brings flexibility and reliability to data operations. The backend is designed with modular routers and controllers, ensuring clean separation of concerns and maintainability. Special attention is given to robust error handling, middleware setup, and security measures like CORS, laying the groundwork for future enhancements such as payment integration, user authentication, and real-time notifications.

In conclusion, this e-commerce web application is not just a demonstration of technical skill, but also a practical solution ready for business implementation and educational advancement. Its successful completion validates the potential of the MERN stack in building high-performance, maintainable, and user-focused web applications for digital commerce.